Kavya Chowdary Immanni

USA • +1 (314) 486-2684 • ikavyach@gmail.com • linkedin.com/in/kavya-immanni • github.com/kc-2705

FULL STACK ENGINEER | REACT · NODE.JS · TYPESCRIPT · EXPRESS · SQL

PROFESSIONAL SUMMARY

Full stack engineer with 5 years of experience building React and Node.js applications end to end, from TypeScript front ends and Express REST APIs to SQL data layers, automated tests, deployment and production support. At DSHB (University of Iowa), architected a React, Node.js and SQL Server portal with 60 secured REST endpoints, a 9-module React/TypeScript admin platform, and an LLM tool-calling shopping assistant over 4,684 products. At Optum, built React/TypeScript claims screens and Spring Boot, Kafka and Redis services on a HIPAA-regulated platform running on Azure Kubernetes Service. Backs every change with Jest, Playwright or JUnit tests, fixes problems at the root cause, and works directly with leadership to turn requests into shipped features.

TECHNICAL SKILLS

Front End: React (Hooks, Context API), TypeScript, JavaScript (ES6+), Redux Toolkit, React Query, React Router, Next.js, Vite, Refine, Ant Design, Recharts, HTML5, CSS3/Sass
Back End: Node.js, Express, REST API Design, Socket.IO (WebSockets), Middleware, File Uploads (Multer), OpenAPI, Java 17, Spring Boot, Apache Kafka, Redis
Databases: Microsoft SQL Server, PostgreSQL, MySQL, Oracle, MongoDB, SuiteQL, Schema Design, Parameterized Queries, Indexing, Caching, Query Optimization
Testing & Quality: Jest, Playwright, React Testing Library, Supertest, Cypress, JUnit 5, Mockito, axe-core (WCAG 2.1), Lighthouse, Performance Benchmarking, Code Reviews
Cloud & DevOps: Docker, Azure (AKS, Key Vault), CI/CD Pipelines, Git & GitHub, Render (Auto-Deploy), Linux Server Deployment, Splunk, Grafana, Health Checks, Alerting
Security: JWT Authentication, bcrypt, Role-Based Access Control, Email OTP, Rate Limiting, Helmet, CORS, Input Validation, HIPAA/PHI Protection, OWASP Secure Coding
Domains: Healthcare Claims (EDI X12 837, ICD-10, CPT/HCPCS, NPI, HIPAA), Research E-commerce (NetSuite SuiteCommerce), Software Globalization (i18n, L10n, G11n: Unicode, Locale Formatting, RTL, XLIFF, AST Scanning)
AI & Platforms: LLM Tool Calling, RAG & Hybrid Search, Model Fallback, Prompt Design; NetSuite (SuiteScript 2.x, SuiteCommerce), Google Apps Script, Looker Studio, GA4, Claude Code, GitHub Copilot

WORK EXPERIENCE

Software Developer – Full Stack, Research E-commerce Platform — Feb 2026 – Present

Developmental Studies Hybridoma Bank (DSHB), University of Iowa | Iowa City, IA

Tech: React, TypeScript, Node.js, Express, SQL Server, Socket.IO, Jest, Playwright, NetSuite, LLM APIs

- Replaced email-and-spreadsheet antibody submissions from research labs worldwide by architecting the Depositor Portal, a React single-page app over a layered Node.js/Express REST API and Microsoft SQL Server, which I own from design through production.
- Moved the entire intake workflow into one system by designing 60 Node.js/Express REST endpoints for submissions, drafts, bulk upload, file upload and download, admin review, pricing rules, history, notifications and exports.
- Secured all 60 endpoints with JWT authentication, bcrypt password hashing, email OTP verification, role-based authorization (depositor vs. admin), rate limiting and Helmet security headers, with matching route guards in React.
- Delivered the portal UI as 30 React components, including a multi-section deposit form, bulk Excel/CSV upload with header normalization, admin dashboard and pricing-rules editor, organized with Context/Provider state, custom hooks and a shared API helper.
- Eliminated manual re-keying of catalog data by building a Node.js export engine that turns approved deposits into NetSuite-ready CSVs (item records, up to 8 pricing levels, matrix child products, categories, collections) with catalog business rules built in.
- Launched every new catalog product search-ready with no manual SEO work by having the export engine generate keyword-rich page titles, meta descriptions and site-search keywords from each antibody's gene, antigen, host and validated applications.
- Let admins see new submissions and status changes the moment they happen, without refreshing or polling, by adding real-time notifications with Socket.IO over WebSockets.
- Brought automated testing to the portal with 50 Jest unit tests on the export engine (82% line coverage) and a benchmark showing ~41,000 rows per second, generating 1,000 products in 98 ms.
- Protected public pages and role-based access in Chrome and Edge with 22 Playwright end-to-end tests, and built axe-core WCAG 2.1 A/AA accessibility scans into the same suite.
- Confirmed a fast front end with a Lighthouse audit (desktop Performance 97, Best Practices 96, SEO 92, 1.2 s largest contentful paint) and turned the mobile results into a plan to cut 202 KiB of unused JavaScript and ~519 KiB of images.
- Gave operations staff self-serve tools by building a 9-module internal admin platform in React, TypeScript, Vite, Refine, Ant Design and Recharts inside NetSuite, backed by SuiteScript JSON APIs that use SuiteQL and server-side caching.
- Restored accurate executive reporting across 190,200 orders and $36.0M in revenue by tracing a 12× revenue-inflation bug and duplicate rows to a stale aggregate cache and a forced-reload defect, then fixing both.
- Replaced a manual "saved search, Excel, import" discount process with a React rule-based audience builder that shows live counts across 57,229 customers and creates NetSuite customer groups for promotions automatically.
- Prevented a production outage of the public blog by catching in sandbox that a platform update broke the CORS preflight, replacing jQuery AJAX with fetch(), and patching sandbox and production before the update went live.

- Gave researchers reliable answers about 4,684 antibodies by designing and shipping an AI shopping assistant in Node.js with LLM tool calling, where every product, price and policy in an answer comes from tool results, not model memory.
- Kept the assistant available through rate limits and failures with a 3-model fallback chain, retry-after handling, an adapter for malformed tool calls, per-visitor rate limits, and twice-daily catalog syncs that never overwrite the last good data.
- Gave staff full visibility into the assistant by logging every chat, rating, contact request and data refresh to Google Sheets through a secret-protected webhook with failure alerts, and shipped it to Render with auto-deploy from GitHub and a health check.
- Restored 493 broken partner datasheet links through a validated bulk import, cleaned 130+ dead links with formula-driven mass updates, and maintain SuiteCommerce storefront extensions in JavaScript, Backbone and Sass.
- Removed manual sales exports for leadership with a SuiteQL to Google Sheets to Looker Studio pipeline, and found that ~701k GA4 users in 28 days were bot traffic, protecting marketing decisions from distorted data.
- Serve as the primary technical contact for the DSHB Director and operations staff, turning requests into specs with trade-offs, testing every change in sandbox with dated backups, and deploying the portal to a university Linux server.

Software Engineer – Healthcare Claims Platform **Aug 2023 – Jan 2026**

Optum, Inc. | Remote

Tech: Java 17, Spring Boot, React, TypeScript, Node.js, Kafka, Redis, EDI 837, Docker, AKS, Splunk

- Kept high-volume healthcare claims moving through intake, validation, eligibility and adjudication by developing and maintaining Java 17/Spring Boot microservices that process EDI X12 837 claim transactions on a cloud-native claims platform.
- Gave claims operations teams faster tools for daily work by developing React and TypeScript screens for claim search, claim history and pended-claim work queues.
- Made large claim-result tables load and scroll faster by adding list virtualization and memoization in React, keeping screens responsive on big result sets.
- Caught invalid claims before they reached downstream adjudication by implementing validation rules for ICD-10 diagnosis, CPT/HCPCS procedure and NPI provider codes.
- Gave internal operations applications dependable claim data by building claim status and history REST APIs with pagination, filtering and OpenAPI documentation.
- Cut repeated calls to downstream member services and sped up claim checks by adding Redis caching for eligibility and reference-data lookups, so the same member and code data was not fetched again for every claim.
- Let claims services deploy independently by building Kafka producers and consumers with Avro schemas and backward-compatible schema changes.
- Reduced manual reprocessing of failed claims by adding idempotency checks, retry topics with exponential backoff and dead-letter handling to Kafka message flows.
- Kept claims flowing on time during peak intake periods by tuning Kafka consumer and partition settings to reduce consumer lag, so events were processed as they arrived instead of piling up behind slow consumers.
- Lessened the platform's dependence on a nightly batch window by helping move a scheduled batch claims workflow to event-driven processing on Kafka, so claims were handled as they arrived rather than once a night.
- Made deployments repeatable and self-healing by containerizing services with multi-stage Docker builds and configuring resource limits, readiness and liveness probes, horizontal scaling and rolling updates on Azure Kubernetes Service.
- Removed embedded credentials and passwords from service code and configuration by moving application secrets to Azure Key Vault and having services access them through managed identities.
- Shipped changes safely through CI/CD pipelines that build, test, scan and deploy containerized services, with health-check-based rollback.
- Helped resolve production incidents faster by tracing claim-processing issues with Splunk logs, correlation IDs and Grafana dashboards, moving from shadowing senior engineers to handling assigned on-call issues and maintaining runbooks.
- Raised test coverage on claims services by writing unit and integration tests with JUnit 5 and Mockito for validation rules, APIs and Kafka message handling, catching defects before they reached production.
- Protected patient health information under HIPAA by masking sensitive data in logs and applying encryption, role-based access and audit logging.
- Turned payer rules into working validation and adjudication logic by partnering with business analysts and claims operations teams to clarify each rule, its edge cases and the expected claim outcome.
- Delivered features in two-week Agile sprints, reviewing teammates' pull requests and documenting APIs in Confluence so other teams could integrate without hand-holding.

Associate Globalization Developer – Internationalization (i18n) & Localization **Aug 2021 – Nov 2022**

TIBCO Software Inc. | Hyderabad, India

Tech: Java, Spring Boot, Spring Data JPA, Oracle, SQL Server, TIBCO EMS/JMS, XLIFF, JavaParser, ESLint, Jenkins, AWS

- Delivered Java/Spring Boot services and REST APIs for MOLEE, TIBCO's enterprise translation management platform, automating the flow of product strings between internal build systems and outside translation vendors across ~10 languages.
- Kept translation projects running smoothly as a technical administrator of the translation management system, configuring project settings, file parsing rules, webhooks and custom connectors to translation vendors' APIs.
- Restored the localization pipeline quickly whenever it broke, fixing webhook payloads that arrived in unexpected formats, XLIFF files that failed to parse on special characters, and job backlogs that delayed translations.
- Caught internationalization bugs in Java and JavaScript/React code before translation by writing AST-based scanners (JavaParser, ESLint) that flag hard-coded strings and string concatenation, and by reviewing code for i18n issues.

- Stopped broken translations from reaching product builds by building Java validators for XLIFF, JSON and .properties resource files that checked structure, required keys and UTF-8 encoding before files were merged.
- Resolved ~10–15 internationalization defects a week with QA, including character-encoding corruption and locale-specific date, time and number formatting errors, by debugging code, logs and data to the root cause and verifying each fix with tests.
- Made products usable in right-to-left languages by implementing and testing Arabic RTL support, including mirrored layouts, bidirectional text and correct alignment, so Arabic releases shipped without layout defects.
- Replaced hard-coded formats with locale-aware Java APIs (Locale, ResourceBundle, DateTimeFormatter, NumberFormat), so dates, numbers and currencies displayed correctly in every supported market without code changes per language.
- Kept high-volume translation jobs fast and stable during release crunches with multithreaded processing on thread pools, JVM memory tuning, and removal of redundant Spring Data JPA queries on Oracle and SQL Server.
- Made failed localization jobs recoverable without losing data by building TIBCO EMS/JMS message processing with client acknowledgment, error queues and replay.
- Automated localization builds and deployments with Maven, Jenkins pipelines and Bash scripts deploying to Docker on AWS, with JUnit tests covering business logic, file validation and error cases.
- Gave localization specialists and support teams self-serve answers by documenting i18n guidelines, integration specs and troubleshooting runbooks in Confluence, and turned their requests into technical tasks in Agile sprints.

PROJECTS

Job Hunt Copilot – AI Web App (DEV Challenge Entry) | React, Node.js, Express, MongoDB, LLM APIs (RAG), Transformers.js | Live · GitHub

- Showed job seekers exactly how well they fit a role by building an LLM pipeline that extracts a posting's requirements and matches each one to resume bullets with hybrid search (embeddings plus keywords), validating every model output in code.
- Kept the live React/Node.js app dependable with 39 automated tests (Node test runner, Supertest, in-memory MongoDB) covering auth, data isolation and AI edge cases, and published a DEV article on the design.

Playverse – 3D Game Hub | React, TypeScript, Vite, Ant Design, Babylon.js, Node.js, Express, Socket.IO, MongoDB, PWA | Live · GitHub

- Built and deployed a full-stack 3D game platform with 4 playable games, accounts, leaderboards and saved progress, installable as a mobile PWA and an Android app, with responsive layouts for desktop and phone.
- Ran real-time multiplayer hide-and-seek on a server-authoritative Socket.IO game server covered by 11 server tests, with enemy AI (vision cones, A* path finding) across 5 themed worlds.

WaitForIt – AI Shopping Agent (PayPal AI Hackathon, Devpost) | Node.js, Express, MongoDB, React, PayPal Orders API, LLM APIs

- Built an AI agent that turns a plain-language wish into a product search, ranks matches with an LLM and places a PayPal authorization hold for the most the user is willing to pay.
- Captured only the real price when it dropped and released or voided the rest of the hold, securing the API with JWT, Zod validation and rate limiting and covering it with 21 API tests.

CERTIFICATIONS & PUBLICATIONS

- AI Application Development with Gemini & Claude (Udemy, 2026); Advanced Design Patterns: Design Principles and Learning SOLID Programming Principles (LinkedIn Learning, 2026)
- Publication: "Job Hunt Copilot: Stop Reading Job Posts You Can't Apply To", DEV Community article that earned the Hacktoberfest 2026 Challenge Completion badge (October 2026)

EDUCATION

M.S. in Computer Science | GPA: 3.86/4.0 **Dec 2024**
Saint Louis University, St. Louis, Missouri, USA

B.Tech in Computer Science | GPA: 3.68/4.0 **Jul 2021**
Gokaraju Rangaraju Institute of Engineering & Technology, Hyderabad, India